2022 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT) | 978-1-6654-5126-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/IAICT55358.2022.9887525

DOTA 2 Win Loss Prediction from Item and Hero Data with Machine Learning

Stanlly
Computer Science Department,
Faculty of Computing and Media,
Bina Nusantara University,
Jakarta, Indonesia 11480
stanlly@binus.ac.id

Fauzan Ardhana Putra
Computer Science Department,
Faculty of Computing and Media,
Bina Nusantara University,
Jakarta, Indonesia 11480
fauzan.putra003@binus.ac.id

Nunung Nurul Qomariyah
Computer Science Department,
Faculty of Computing and Media,
Bina Nusantara University,
Jakarta, Indonesia 11480
nunung.qomariyah@binus.ac.id

***Abstract*—Video gaming has become a titan in the overall market over the past decade, culminating in an estimated worth almost 180 billion US dollars by 2021. Aside from its growing influence in the overall market, video games have also created a new competitive format called eSports, a format where highly skilled players of certain video games play against each other in a tournament to see who the most skilled are and win a prize at the end. ESports are just one of many reasons why people have become interested in the idea of being able to predict the outcome of any given match between players. In this study, We conducted research on the importance of certain factors in determining the win or loss of any given Defense of the Ancients 2, better known as DOTA 2, match. We found that Item and Hero choices play a large role in winning any given match. From this, we concluded that we would be able to predict a match's outcome solely based off of these two factors and created models to predict the outcome of any given match. In this study, we will be employing the use of Decision Tree, Random Tree and XGBoost classifiers in order to create our models. In the end, the XGBoost model ended up being our best model, with an accuracy of roughly 93% which can predict an outcome in roughly one minute.**

***Index Terms*—Game, win prediction, classification, machine learning, tree based model**

I. Introduction

Over the past decade, the video game industry has continuously surprised everyone with its rapid growth. Coming from its niche roots in the arcades back in the 1970s to becoming even bigger than the movie and music industry combined by 2021. With the rise of the video game industry, the creation of a new form of competition: "eSports" was created. ESports are essentially a collection of highly skilled players competing against each other in order to see who is the better player, a similar concept to real life sports. Spanning over hundreds of games and multiple unique genres, one of the most popular eSports game genres is the Multiplayer Online Battle Arena (MOBA). One game in said genre is Defense of the Ancients 2, better known as DOTA 2. As the successor to one of the first major titles in the genre, DOTA 2 has garnered a sizeable fanbase that is still active to this day. Ranking as the 2nd most played game on the game platform Steam. The game is a competition between 2 teams of 5, radiant and dire. Players get a choice of a single hero each, with each hero being able to fill up to six slots with items throughout the match. The goal of each match is to tear down the opposing team's defenses in order to get to and destroy their ancient, a heavily guarded structure. DOTA 2 is considered one of the most lucrative eSports game in the current scene, because of this a lot of people are still interested in being able to predict which team will win or lose. The ability to predict the outcome of a given match could also prove useful in formulating new strategies and determining whether certain combinations of hero and item choices are better than others, a process that is constantly being done as the game evolves.

As avid fans of DOTA 2, we are also curious on whether such a model is possible. With personal knowledge on how the game is structured and a good idea on how each factor plays into the outcome of a match, our team believes that we will be able to create a simple but accurate model that is able to predict the outcome of any given match with ease. We also believe that the ability to predict match outcomes based on item and hero combinations can prove useful in creating "builds", combinations of items and heroes that are used to achieve certain play styles or goals, as well as many other things relevant to the DOTA 2 community.

There have been multiple different relevant studies conducted by a plethora of different researchers on predicting the outcome of a DOTA 2 match using many different metrics and factors. Ranging from a study conducted by Grutzik, Higgins and Tran [1] and their 61% accurate team and draft feature model that aimed to take player history into account for their prediction to Akhmedov and Phan's [2], where they used linear regression and neural networks models with 82% and 88% accuracy. The authors of [3], also used four different machine learning models, one of which managed to give up to 85% accurate after 5 minutes of gameplay. Another study conducted by Yang, Qin and Lei [4] tried to create a machine learning model using real-time features which had an accuracy of 93.73% by the 40th minute of any given match. Johansson and co. [5] have also created a similar type of model using random forest classifier model that has an average accuracy of 88.83%.

There are also a few studies that corroborate the idea that item and hero choices play an important role in the outcome of any given match. Such as a study conducted

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:36:13 UTC from IEEE Xplore. Restrictions apply.

by Song and co. [6] which found that trying to make a linear regression model using only heroes as features was entirely not enough to create a good predictive model. Wang [7] also conducted a study on the importance of hero drafts on the outcome of a match which uses a neural network trained model that ended with roughly 61% accuracy. Another study by Agarwala and Pearce [8] also tried to use linear regression to create models based on whether hero selection has an effect on win rate, culminating in a model that is 62% accurate. An interesting study conducted by Kinkade, Jolla and Lim [9] tries to incorporate hero matchup, synergy and counters for their model with a final prediction accuracy of 73%.

While not directly related to our current study, there have also been many other machine learning models made for other aspects of DOTA 2. A few of such examples include Summervile and co.'s model that helps decide strategic choices for heroes during the draft pick phase of a match [10], Conley and Perry's model that helps recommend herores to pick to the player using the model [11] and Katona and co.'s model that tries to accurately predict player deaths within a five-second window [12].

With these studies in mind, we decided to conduct our research as we noticed that a lot of them did not involve some of the currently popular methods of machine learning classifiers, mainly the ones we used in our paper: Decision Tree, Random Forest and XGBoost. While some of the studies do try to use the hero data and real-time match data to create their models, we also noticed that a majority of them do not combine the features of item and hero data in their predictive models.

The goal of this paper is to ascertain the importance of factors unrelated to player skill, such as hero and item choices, in determining the outcome of any given match of DOTA 2. This research will also determine whether certain factors within any given match of DOTA 2 should be given more consideration to while playing the game in order to increase the chances of victory.

II. Methodology

A. Data Collection

In this study, we use a dataset posted by user Devin Anzelmo found on Kaggle[1]. This dataset is an update to a previously posted dataset of the same name. The data is a subset of a data dump created using Opendota, otherwise known as yasp.co [13], which parsed ongoing matches and compiled them into CSV files based on specific parameters[2].

With this dataset, we explore the possibility of creating a predictive model to determine the outcome of any given match through the use of different factors. After a bit of research and experimentation, we determined that the simplest model that we could use to create an accurate model are item and hero data. This dataset is extensive and contains many different CSV files within the overall archive. For our research, we decided to use the CSVs that contain the hero and item names and ids, as well as the match and players CSVs. We chose to focus on using item and hero data alongside the information from the match data as these factors are chosen by the players in a match and are not solely determined through random chance.

[1]https://www.kaggle.com/devinanzelmo/dota-2-matches
[2]https://github.com/odota/core/wiki/JSON-Data-Dump

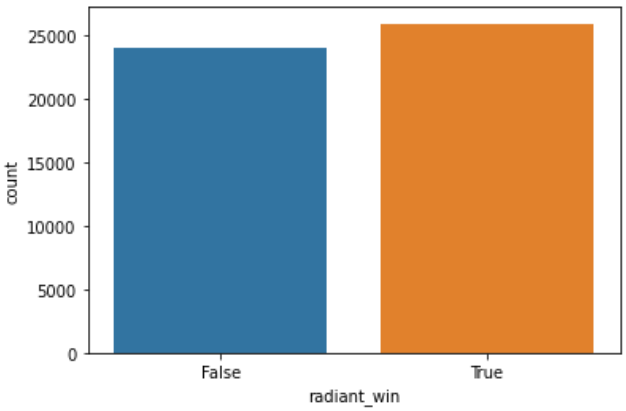


Fig. 1. Counting Radiant Wins and Losses

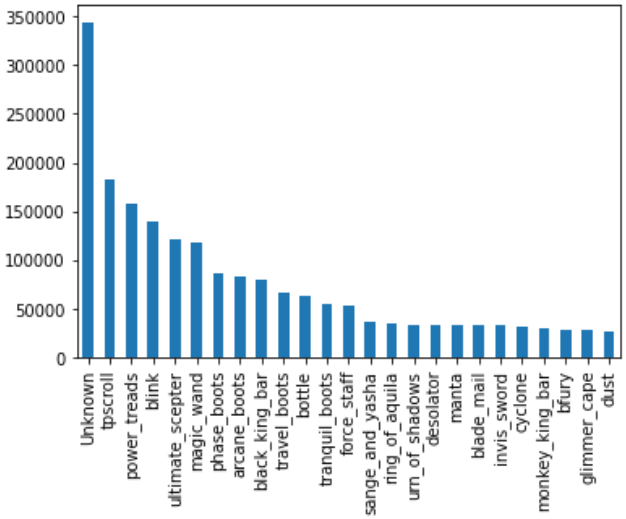


Fig. 2. Top 25 Most Bought Items

In the CSV files that we used, the hero_names.csv and item_ids.csv contain the ID of the hero or item and the names associated with those IDs. Our features from these files will be item_id and hero_id while our label is only radiant_win. With these labels in mind, figure 1 depicts the counts, in booleans, for the column radiant_win, which shows that the dataset contains a roughly equal amount of wins and losses.

Figure 2 depicts the top 25 items owned by the end of all matches. In this figure, we can see that top items is unknown, this is due to the fact that all players start with empty slots, which is then counted towards being a common item in every player's build within the dataset. We can ignore this data as it is essential just dummy data. From the graph, we can see that the top 3 most used items are: tpscroll, power_treads and blink.

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:36:13 UTC from IEEE Xplore. Restrictions apply.

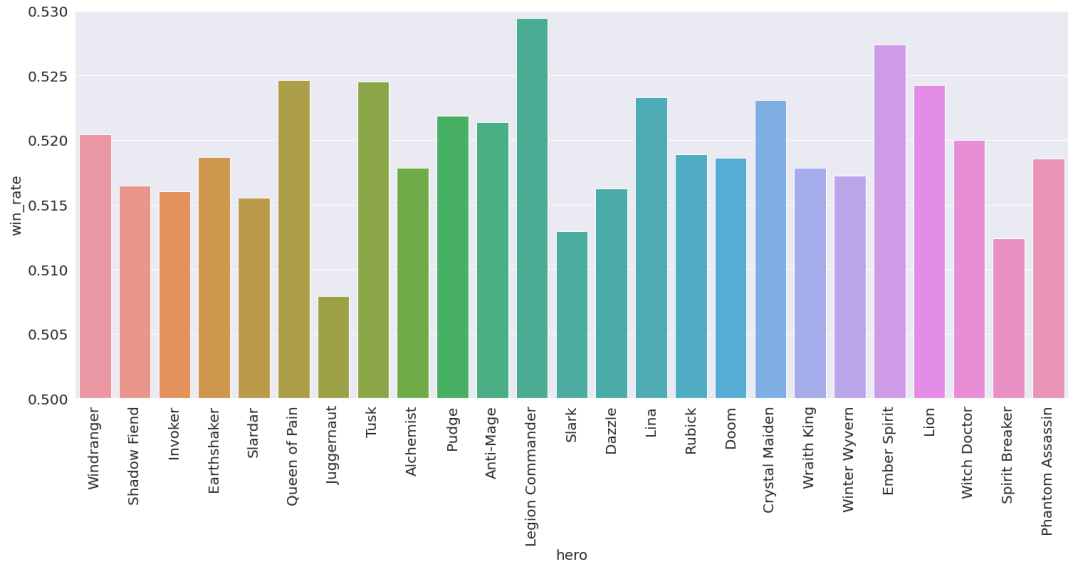


Fig. 3. Top 25 Heroes vs Win Rate

TABLE I
BEFORE BINARY ENCODING

Index	Hero
0	Windranger
1	Shadow Fiend
2	Juggernaut
3	Invoker

TABLE II
AFTER BINARY ENCODING

idx	Windranger	Shadow Fiend	Juggernaut	Invoker
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1

Figure 3 depicts the top 25 heroes chosen by players, shown in order from left to right and their respective win rates, shown by the bar chart. As seen from the graph: the highest picked hero, Windranger, does not show any significantly higher win rate for the team who picked the hero, which shows that picked heroes alone are not a factor that determines the outcome of a match. We can further see this trend as we inspect the graph as Juggernaut: the 7th most picked hero, has a below average win rate compared to the other 25 heroes depicted in the figure.

B. Data Preprocessing

After making sure that all item and hero data are correct and that there are no null values, we started to one-hot encode our data through the use of binary encoding. We do this as we are trying to train our predictive model, which learns better from binary data. The following tables represent the data before and after binary encoding:

As seen from the two tables, we binary encode the data by putting the hero column in the players CSV file through pandas's get_dummies function which turns a categorical column into indicator columns of 0s and 1s. In the table example, we can see that indexed values of hero name by index is turned into a binary value to represent what the hero is. Going from index 1 representing Windranger to giving that value a binary equivalent value of 1000. These new binary values are then stored as a dataset that will be fed into our model for training.

We also did this process to our item data, with a default of 0 in each item slot. The values of these slots are then added into a binary that is then separated during the learning process. Player and hero data are then separated into two teams, this data will also include the previous arrays.

C. Models and Techniques

We decided to use three different classifiers in order to create three different predictive models and find the most accurate model. We decided to use three different algorithms for this paper. The decision tree algorithm: which works by making decisions based on a condition that then branches out into multiple endpoints with their own conclusions. The random forest algorithm: which uses the same base concept as decision tree but adds a bit of randomness while calculating a user-defined cluster of different trees, making each individual tree uncorrelated to each other but able to predict more accurately than an individual calculated tree, and finally, XGBoost: an open-source implementation of the gradient boosted trees algorithm, which can be used for regression or classification tasks. It works through the use of boosting, an ensemble method that combines predictions from several models into one, taking each predictor sequentially and improving itself based on its predecessor's errors by giving more weight to predictors that perform better. It also uses a specific type of boosting known as gradient boosting that tries to minimises the loss function using a gradient descent algorithm. It also uses the base concept of decision trees to determine its "weak" predictors. We

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:36:13 UTC from IEEE Xplore. Restrictions apply.

then trained these models through the use of the following libraries:

- Xgboost to train our XGBoost classifier model.
- Sklearn to train our Decision Tree and Random Forest classifier models.
- Seaborn and matplotlib for visualizations
- Pandas and numpy for number and array processing

Using the data that we previously processed in section II.B, we created a training dataset X through concatenating the hero and item data. We then created dataset y and split the data through a boolean to determine whether team radiant had won the match or not. If the data is false, the team is dire. Dataset X is then simplified through the use of Principal Component Analysis (PCA). The data is currently too complex as it has roughly 612 estimated number of features, PCA is used to shrink this number to only 70 features which will improve the training time as well as the efficiency of our models.

In order to test how accurate our models are, we will be employing two methods of testing: cross validation and holdout testing. For our holdout testing, we will be splitting training and test sets as 80% training and 20% testing datasets, while we use sklearn's built-in cross validation method to test the cross validation score.

We decided that our random_state hyperparameter for all of our models are the same in order to gain some form of consistency between each model, a random_state of 42. Each of our models are also using the following hyperparameters:

- For our Decision Tree Classifier, we decided to set the hyperparameters max_depth as 10.
- Our Random Forest Classifier uses a max_depth of "None" which means that all of our nodes are expanded until all leaves are pure or all leaves contain less than the default min_sample_split of 2. We also set the number of n_estimators, the number of trees in the forest, to 100
- Our XGBoost Classifier uses a max_depth of 6 in order to prevent the model from becoming too complex and overfitting, with a gamma of 0. While setting the eta, learning rate, to 0.3 in order to get the best chances of obtaining an optimal model.

III. Result and Discussion

After training our models, we used the sklearn's built-in cross validation method to determine how accurate our predictive models are. The function is built to evaluate the accuracy of a dataset through the use of different parts of the datasets that our models have not yet seen in order to give a fair assessment on the accuracy of our predictions.

TABLE III
Accuracy of Models Created and Time Taken to Train

Classifier	Precision	Recall	F1	Training Time
Decision Tree	0.82	0.82	0.82	19 seconds
Random Forest	0.91	0.91	0.91	3 minutes
XGBoost	0.94	0.94	0.94	~1 minute

The Decision Tree classifier returned a cross validation score of roughly 82% accuracy with a training time of roughly 19 seconds. Random Forest, on the other hand, returned a cross validation score of roughly 91% accuracy while being trained for 3 minutes. XGBoost is our final and best model of the three we created. Returning an impressive 94% accuracy on the cross validation test and taking only one minute to train. Aside from the cross validation score from sklearn, we also used a few different metrics, including but not limited to: precision, recall and F1 scores as well as how much time was taken to train each model, to measure accuracy in Table III.

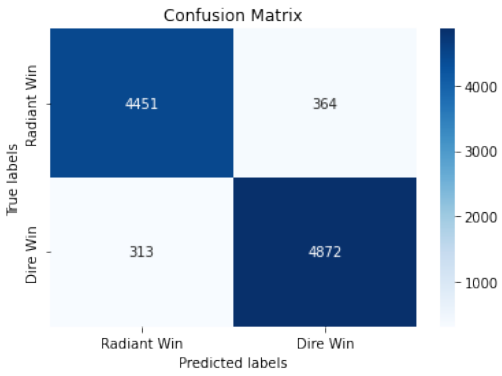


Fig. 4. Confusion Matrix

As XGBoost returned the best results overall, we will be expanding on the overall accuracy and specifics on our best model. Figure 4 also depicts the confusion matrix of our XGBoost model. As seen from the matrix 4, we have a total of 677 false metrics from a total of 10,000 samples for a total of 93.23% true accuracy on our predicted labels.

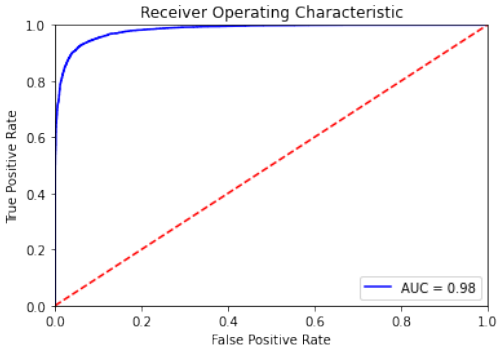


Fig. 5. ROC Curve

We also created a Receiver Operating Characteristic (ROC) curve to visualize our model's accuracy. As seen by the visualization in Figure 5, the curve is very close to 1.0 true positive rate, also known as the ideal clinical discriminator, the closer the curve is to this point the more accurate it is. Our curve is very close to this point and away from the diagonal which shows that our model is very accurate.

Figure 6 is a visualization of the tree created by our XGBoost algorithm model. F* represents the features, as we passed our features through PCA the result is a number from -1 to 1. For example, if f23 is less than -0.2 we then

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:36:13 UTC from IEEE Xplore. Restrictions apply.

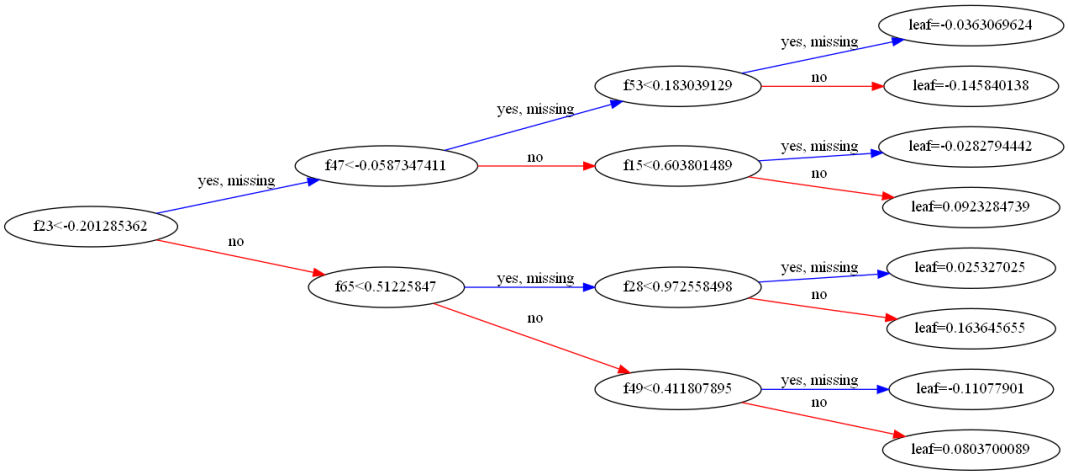


Fig. 6. XGBoost Tree Visualization

check for f47, etc. The following equation can be used to further explain the process:

$$probability = 1/(1 + e^{-value}) \quad (1)$$

For example, if a classification tree has 2 classes 0,1, the value of the leaf node represent the raw score for class 1. It can then be converted to a probability score by using equation 1. By using the equation with the topmost leaf as its value, we get a probability of 0.490924. This value is the probability of the data point being class 1 when being distributed to the topmost leaf.

Now that we know that XGBoost is our final and best model, we decide that we should conduct a test to determine if our model is overfitted. As our model was able to accurately predict the outcome of a match, we will assume that the model is not underfitted. We then used the classification report method that comes with sklearn in order to check for the presence of overfitting. After our test, we concluded that our model has been overfitted. After analyzing our workflow and our implementation, we boiled down the possible reasons of why our data became overfitted to the following reasons. The first reason is that we failed to standardize the data before starting to train our model. We may have also failed to use any form of data augmentation to regularize the training data in order to avoid the overfit. We also did not include a stopping mechanism to limit the amount of time the model had spent while training, thus causing an overfit due to training for too long. The final possible reason is that our model is simply too complex.

IV. Conclusion and Future Work

From our models, we can conclude that among many other factors, hero and item choices are the two most important factors in any given match of DOTA 2. These factors are the most notable as they are completely unrelated to the skill of any given player, making them a good way to increase the certainty of victory for those who are looking to improve their win rate. This means that players should pay close attention to not only their own hero and item choices, but also the choices of the enemy team in order to maximize their potential to win the match.

Out of all the related works stated, our model seems to be decently accurate in predicting the outcome of a match. Comparing it to Akhmedov and Phan's [2] 82% and 88% models, our XGBoost model managed to be more accurate than both models: with an accuracy of 93.23%. Our best model is also only slightly less accurate than Yang, Qin and Lei's model at 93.73%.

We will likely be remaking a new model in the future as there are many things that we could have done to improve our models. Normalizing and regularizing the data before training will be essential in creating a more accurate model that does not experience overfitting. We would have also benefited more from planning out which data points we wanted to use and reasons for using said data in order to create a more accurate model that can incorporate more factors into predictions as we were only able to incorporate item and hero data with our knowledge at the time of creating this model. We should also try to make a better model for active predictions of ongoing matches as the ability to predict if a finished match is a win or loss is redundant in hindsight, though we would be able to make a graphical display to figure out a match's outcome through item and hero data at any given point of the match to make it more practical for predicting ongoing match outcomes in order to improve our current model.

Supplementary Source Code

To see the source code please head to the following link: https://github.com/fauzanardh/DotaMatchPredictor

References

[1] P. Grutzik, J. Higgins, and L. Tran, "Predicting outcomes of professional dota 2 matches," Technical Report. Stanford University, Tech. Rep., 2017.

[2] K. Akhmedov and A. H. Phan, "Machine learning models for dota 2 outcomes prediction," *arXiv preprint arXiv:2106.01782*, 2021.

[3] V. J. Hodge, S. M. Devlin, N. J. Sephton, F. O. Block, P. I. Cowling, and A. Drachen, "Win prediction in multi-player esports: Live professional match prediction," *IEEE Transactions on Games*, 2019.

[4] Y. Yang, T. Qin, and Y.-H. Lei, "Real-time esports match result prediction," *arXiv preprint arXiv:1701.03162*, 2016.

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:36:13 UTC from IEEE Xplore. Restrictions apply.

[5] F. Johansson and J. Wikström, "Result prediction by mining replays in dota 2," 2015.
[6] K. Song, T. Zhang, and C. Ma, "Predicting the winning side of dota2," *Sl: sn*, 2015.
[7] W. Wang, "Predicting multiplayer online battle arena (moba) game outcome based on hero draft data," Ph.D. dissertation, Dublin, National College of Ireland, 2016.
[8] A. Agarwala and M. Pearce, "Learning dota 2 team compositions," *Sl: sn*, 2014.
[9] N. Kinkade, L. Jolla, and K. Lim, "Dota 2 win prediction," *Univ Calif*, vol. 1, pp. 1–13, 2015.
[10] A. Summerville, M. Cook, and B. Steenhuisen, "Draft-analysis of the ancients: predicting draft picks in dota 2 using machine learning," in *Twelfth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2016.
[11] K. Conley and D. Perry, "How does he saw me? a recommendation engine for picking heroes in dota 2," *Np, nd Web*, vol. 7, 2013.
[12] A. Katona, R. Spick, V. J. Hodge, S. Demediuk, F. Block, A. Drachen, and J. A. Walker, "Time to die: Death prediction in dota 2 using deep learning," in *2019 IEEE Conference on Games (CoG)*. IEEE, 2019, pp. 1–8.
[13] A. Cui, H. Chung, and N. Hanson-Holtry, "Yasp 3.5 million data dump."

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:36:13 UTC from IEEE Xplore. Restrictions apply.